



FACULTAD DE CIENCIAS DE LA VIDA Y TECNOLOGÍAS

Documento Técnico: Módulo Parqueo y Vehiculos

Nombres:

Winter Manuel Povea Bravo

Docente:

Ing. John Cevallos

Asignatura:

Aplicaciones Web II

Curso:

6to “A”

Periodo:

2026-1

Índice

1.	Introducción.....	2
2.	Problema 1: Diseño e implementación de los handlers HTTP	2
3.	Problema 2: Centralización de respuestas HTTP mediante respond.go	2
4.	Problema 3: Diseño de una interfaz para desacoplar la lógica de almacenamiento .	3
5.	Reflexión sobre la programación en Go	3
6.	Link del repositorio con el video	4

1. Introducción

Durante el desarrollo del Módulo de Parqueo y Vehículos para el Sistema de Gestión de Condominios se implementó una API REST en Go utilizando el router Chi y almacenamiento en memoria. El módulo permite administrar vehículos de residentes, visitas de vehículos externos y la bitácora de accesos vehiculares. Durante el proceso se presentaron diversos desafíos técnicos relacionados con la organización del código, la reutilización de componentes y el modelado de reglas de negocio.

2. Problema 1: Diseño e implementación de los handlers HTTP

Uno de los principales desafíos fue comprender la estructura de los handlers y la forma en que las solicitudes HTTP se transforman en operaciones sobre los datos almacenados.

Cada endpoint debía recibir solicitudes del cliente, interpretar parámetros de la URL, validar información enviada en formato JSON y generar respuestas adecuadas. Inicialmente la lógica tendía a repetirse en múltiples funciones, especialmente en la lectura de identificadores y el manejo de errores.

Para solucionar este problema se incorporó una función auxiliar denominada `parseID`, encargada de obtener el parámetro `id` desde la URL y convertirlo a un valor numérico seguro. Esto permitió evitar la duplicación de código en múltiples endpoints y mejorar la mantenibilidad del proyecto.

Además, se implementó una función centralizada denominada `MontarRutasParqueo`, la cual registra todas las rutas pertenecientes al módulo. Esta estrategia facilita la integración con otros módulos del proyecto y reduce conflictos cuando varios desarrolladores trabajan simultáneamente sobre el mismo repositorio.

3. Problema 2: Centralización de respuestas HTTP mediante `respond.go`

Otro reto importante consistió en evitar la repetición de código relacionada con las respuestas HTTP.

En una implementación inicial, cada handler debía configurar manualmente los encabezados HTTP, establecer códigos de estado y serializar los datos en formato JSON. Este enfoque aumentaba la cantidad de código repetido y elevaba el riesgo de inconsistencias.

Para resolver esta situación se creó el archivo `respond.go`, que contiene las funciones `RespondJSON` y `RespondError`. Estas funciones encapsulan la lógica necesaria para construir respuestas exitosas y respuestas de error de forma uniforme.

Como resultado, todos los endpoints generan respuestas con la misma estructura, facilitando tanto el mantenimiento del código como el consumo de la API desde aplicaciones cliente. Esta práctica se aproxima a los estándares utilizados en proyectos profesionales de desarrollo backend.

4. Problema 3: Diseño de una interfaz para desacoplar la lógica de almacenamiento

Uno de los aspectos más complejos fue comprender el uso de interfaces en Go y aplicarlas al almacenamiento de datos.

Se diseñó la interfaz `AlmacenParqueo`, la cual define todas las operaciones que debe implementar cualquier sistema de persistencia relacionado con el módulo de parqueo. Posteriormente se desarrolló la estructura `MemoriaParqueo`, encargada de almacenar temporalmente la información en memoria mediante slices.

La principal ventaja de este enfoque es el desacoplamiento entre los handlers y la implementación concreta del almacenamiento. Los handlers interactúan únicamente con la interfaz y desconocen si los datos provienen de memoria, una base de datos SQL o cualquier otro mecanismo de persistencia.

Esta decisión arquitectónica facilita futuras migraciones hacia tecnologías como PostgreSQL o MySQL sin necesidad de modificar la lógica de negocio ya implementada.

5. Reflexión sobre la programación en Go

La experiencia de desarrollar este módulo permitió comprender varias características distintivas del lenguaje Go. Una de las más destacadas es su simplicidad sintáctica, la cual favorece la lectura y mantenimiento del código.

Asimismo, el uso de interfaces proporciona un mecanismo poderoso para desacoplar componentes sin recurrir a estructuras excesivamente complejas. También resultó interesante observar cómo Go promueve la organización modular mediante paquetes claramente definidos.

Finalmente, se evidenció la importancia de diseñar correctamente las responsabilidades de cada componente. La separación entre modelos, handlers, almacenamiento y configuración del servidor permitió construir una solución más ordenada, escalable y fácil de mantener.

En conclusión, Go demostró ser una alternativa eficiente para el desarrollo de APIs REST, ofreciendo un equilibrio adecuado entre simplicidad, rendimiento y buenas prácticas de ingeniería de software.

6. Link del repositorio con el video

https://drive.google.com/file/d/1ja_g05Y5Fqoic-0DauSwpkENuVwKVj9I/view?usp=sharing